

Algorithmen und Datenstrukturen | AlgDat

Zusammenfassung

INHALTSVERZEICHNIS

1. Suchbäume	2
1.1. Binärer Suchbaum	2
1.1.1. Suche	2
1.1.2. Einfügen	2
1.1.3. Löschen	2
1.1.4. Performance	2
1.1.5. Arithmetische Progression	2
2. Sorting	3
3. Text Processing	3
4. Graphs	3

1. SUCHBÄUME

Multimaps: Ungeordnet, `find(k)`, `findAll(k)`, `insert(k,o)`, `remove(e)`

Geordnete Multimaps: Geordnet, `first()`, `last()`, `successors(k)`, `predecessors(k)`

Binäre Suche: Bei einer Multimap, realisiert als array-basierte Sequenz, sortiert nach Key, bei `find(k)`: bei jedem Schritt wird die Anzahl der Kandidaten halbiert, terminiert nach $O(\log n)$ Schritten

Suchtabelle: Multimap, welche mithilfe einer sortierten Sequenz implementiert wird.

- `find`: $O(\log n)$
- `insert`: $O(n)$
- `remove`: $O(n)$

1.1. BINÄRER SUCHBAUM

Ein binärer Baum, welcher Keys (oder Key- Value-Entries) in seinen internen Knoten speichert und folgende Bedingungen erfüllt:

Gegeben sind die drei Knoten u , v , und w . u ist im linken Teilbaum von v und w ist im rechten Teilbaum von v . So gilt: $\text{key}(u) \leq \text{key}(v) \leq \text{key}(w)$

TODO:

u links, v oben, w rechts

TODO:

OOP2 inorder etc traversing

1.1.1. Suche

TODO:

OOP2, slides 8

1.1.2. Einfügen

TODO:

OOP2, slides 9

1.1.3. Löschen

TODO:

OOP2, slides 11

1.1.4. Performance

Betrachte eine Map mit n Entries, welcher mit einem binären Suchbaum implementiert ist mit der Höhe h

- nötiger Speicher ist $O(n)$
- Methoden `find`, `insert` und `remove` benötigen $O(h)$
- Die Höhe h ist $O(n)$ im schlechtesten Fall und $O(\log n)$ im besten Fall

1.1.5. Arithmetische Progression

Der Worst-Case tritt z.B. ein, wenn man aufsteigend sortierte Werte in einen binären Such-Baum einfügt. Laut der **Gauss'schen Summenformel** $\frac{n(n+1)}{2}$ ist die Worst-Case Laufzeit somit $O(n^2)$.

TODO:
implementation?

2. SORTING

3. TEXT PROCESSING

4. GRAPHS